New issue

[Feature]: Support Persistent/Pinned Prefixes in Prefix Caching #23083

Closed as not planned

Labels feature request stale

kangyishuai opened on Aug 18, 2025 ...

🚀 The feature, motivation and pitch

I'm writing to propose a feature enhancement for the Prefix Caching mechanism. Currently, vllm uses an LRU (Least Recently Used) eviction policy for prefix caching: when the cache reaches its capacity limit, the least recently accessed prefixes are evicted to make space for new ones. This works well for general use cases, but there are scenarios where users might need **specific critical prefixes to remain persistently in memory (or VRAM) and never be evicted**, even if they are not the most recently used.

Use Case Motivation:

In production environments, certain prefixes (e.g., system prompts, common instruction templates, or frequently reused context chunks) are accessed repeatedly across many inference requests. Evicting these prefixes can lead to unnecessary recomputation when they are needed again, which undermines the performance benefits of caching. Allowing users to "pin" such critical prefixes would ensure they remain in cache indefinitely, optimizing latency for high-priority workloads.

Proposed Enhancement:

Add a mechanism to mark specific prefixes as "pinned" (non-evictable). Pinned prefixes would:

1. Be prioritized for retention in cache, even when the cache is full.
2. Not be subject to LRU eviction, regardless of their access frequency.
3. Occupy a reserved portion of the cache (or be counted against the total capacity but excluded from eviction logic).

This could be exposed via an API parameter (e.g., `pin_prefix=True` when submitting requests) or a configuration setting to specify which prefixes should be pinned.

Potential Considerations:

- Ensuring pinned prefixes do not exceed the total cache capacity (with safeguards/errors if users attempt to pin more than possible).
- Balancing pinned and dynamic (evictable) prefixes to avoid underutilizing cache space.

Alternatives

No response

Additional context

No response

Before submitting a new issue...

- [x] Make sure you already searched for relevant issues, and asked the chatbot living at the bottom right corner of the documentation page, which can answer lots of frequently asked questions.

👍 4

kangyishuai added **feature request** on Aug 18, 2025

ZJY0516 on Aug 19, 2025 Member

I'm curious—do you have any workload data or performance logs demonstrating that pinned prefixes can effectively reduce latency?

luolun mentioned this on Aug 26, 2025

[RFC]: Frequency and Cost Aware Eviction Policy for Prefix Caching #23641

vllmellm mentioned this on Aug 26, 2025

[Feature]: Frequency and Cost Aware Eviction Policy for Prefix Caching vllmellm/vllm#20

kangyishuai on Aug 28, 2025 Author

> I'm curious—do you have any workload data or performance logs demonstrating that pinned prefixes can effectively reduce latency?

Yes, when I'm working on NL2SQL, there are many prefixes that need to be entered every time, and quick response is required. With the prefix cache, the response speed has increased significantly. This feature has been very helpful for my business. However, I'm worried that the cache might be overwritten, so I have a need for fixed cache.

dongbo910220 on Sep 17, 2025 · edited by dongbo910220 Edits Contributor

Hi, I'm interested in tackling this feature. This issue is currently unassigned. If no one is actively working on it, I would be happy to pick it up and start working on a PR.

👍 1

dongbo910220 mentioned this on Oct 17, 2025

feat(v1): Implement pinned prefix caching for V1 engine #27046

dongbo910220 added 2 commits that reference this issue on Oct 17, 2025

`feat(v1): Implement pinned prefix caching for V1 engine ...` 93c7372

dongbo910220 mentioned this on Oct 17, 2025

feat(v1): Implement pinned prefix caching #27097

liu-cong on Nov 8, 2025

This is an interesting feature, however it adds operational overhead as well, I think we should think through the lifecycle a bit more. A few questions:

- How to "unpin" a cache?
- What are the scenarios that the engine will force evict a "pinned" cache?

yeqcharlotte added this to Docker Image Optimization on Nov 13, 2025

github-project-automation moved this to Todo in Docker Image Optimization on Nov 13, 2025

yeqcharlotte removed this from Docker Image Optimization on Nov 13, 2025

github-actions bot on Feb 6 – with GitHub Actions

This issue has been automatically marked as stale because it has not had any activity within 90 days. It will be automatically closed if no further activity occurs within 30 days. Leave a comment if you feel this issue should remain open. Thank you!

github-actions added **stale** on Feb 6

github-actions bot on Mar 8 – with GitHub Actions

This issue has been automatically closed due to inactivity. Please feel free to reopen if you feel it is still relevant. Thank you!

github-actions closed this as not planned on Mar 8

Metadata

Assignees

No one assigned

Labels

feature request **stale**

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

.

None yet

Development

No branches or pull requests

Participants

 +1